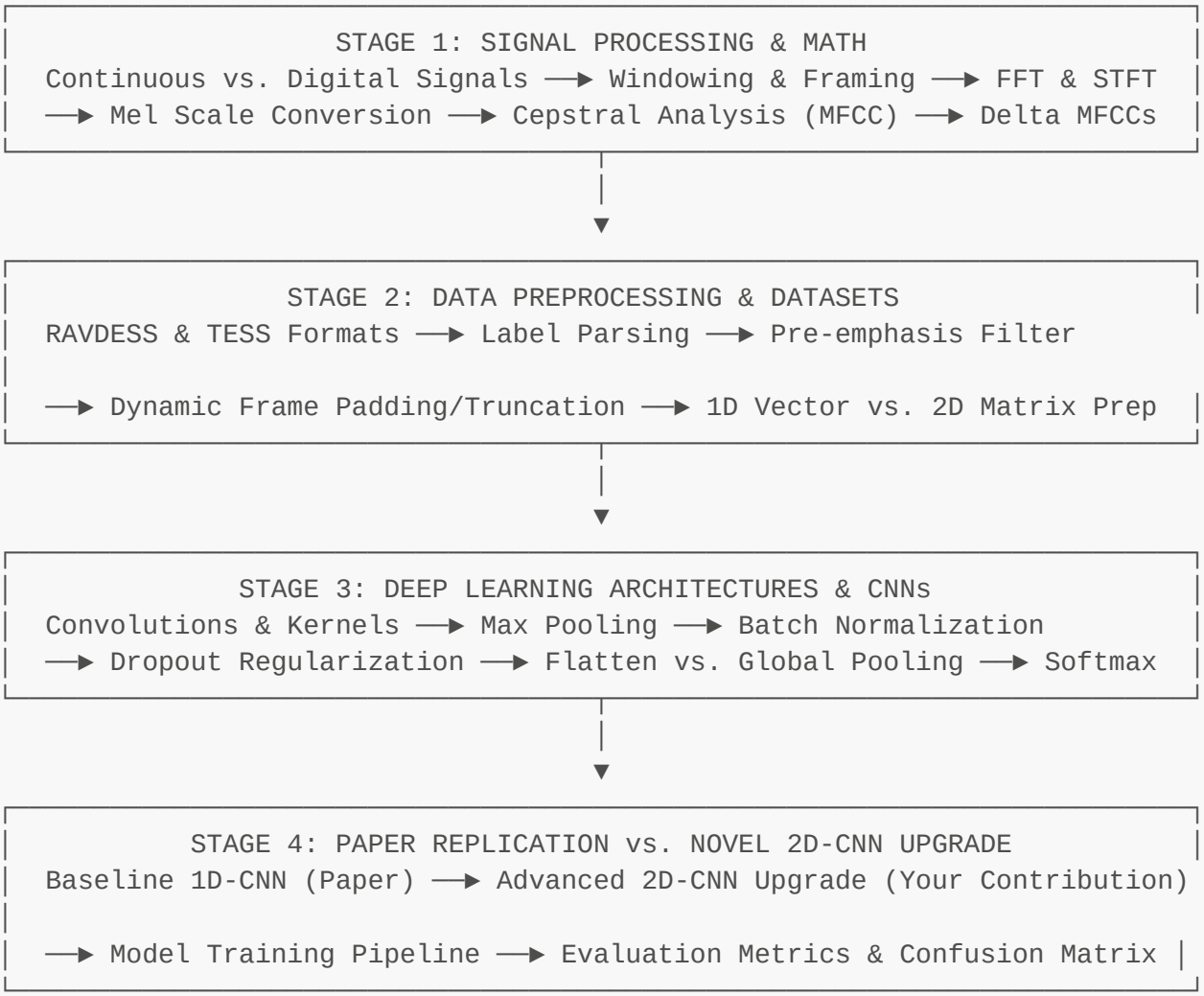
Complete Master Roadmap Overview



STAGE 1: Signal Processing & Math Fundamentals

1. Continuous Audio to Digital Signal Conversion

Sound is a continuous air pressure wave $s(t)$. To process it on a computer:

- **Sampling:** Converting continuous time t into discrete time indices n at a **sampling rate** f_s (e.g., $f_s = 16,000 \text{ Hz} = 16,000 \text{ samples/second}$).
- **Nyquist-Shannon Theorem:** The maximum frequency captured accurately without aliasing distortion is $f_{\max} = \frac{f_s}{2}$. For 16 kHz audio, the maximum detectable audio frequency is 8 kHz.

2. Stationarity, Framing, and Windowing

Speech is **non-stationary** (its frequency spectrum changes constantly over time). However, across tiny temporal intervals ($20–30 \text{ ms}$), speech remains quasi-stationary.

- **Framing:** We slice continuous audio array $x[n]$ into overlapping frames:
- **Window Size (N):** $25\text{ ms} \times 16,000\text{ Hz} = 400\text{ samples}$ per frame.
- **Hop Size (M):** $10\text{ ms} \times 16,000\text{ Hz} = 160\text{ samples}$ shift between adjacent frames.
- **Window Function (Hamming Window):** Slicing introduces sharp signal discontinuities at frame edges (spectral leakage). A Hamming window smoothly tapers the edges to zero:

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \le n \le N-1$$

3. Fourier Transforms: FFT and STFT

- **Discrete Fourier Transform (DFT):** Converts a time-domain frame $x[n]$ into its component frequency bins $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot w[n] \cdot e^{-j\,\frac{2\pi}{N}kn}$$

- **Fast Fourier Transform (FFT):** An optimized algorithm calculating the DFT in $\mathcal{O}(N \log N)$ time using $N = 512$ Fourier coefficients.
- **Short-Time Fourier Transform (STFT):** Running the FFT across every consecutive frame yields a **2D Spectrogram** (Frequency bins $\times$ Time frames).

4. Human Auditory Perception: The Mel Scale

Humans perceive sound pitch logarithmically. We easily detect a 100 Hz change at low pitch (200 Hz vs. 300 Hz), but cannot distinguish 100 Hz changes at high pitch (8000 Hz vs. 8100 Hz).

- **Mel Warping Equation:** Converts physical frequency f in Hertz into perceptual Mel pitch m:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

- **Mel Filter Bank:** A set of overlapping triangular bandpass filters ($M = 24$–40 filters) spaced evenly on the Mel scale to compress raw FFT frequency bins into perceptually weighted energy values.

5. Vocal Tract Filtering: Cepstral Analysis & MFCCs

Speech production follows the **Source-Filter Model**:

- **Source ($e[n]$):** Glottal pulses from vocal cords (pitch/excitation).
- **Filter ($h[n]$):** Vocal tract shape (mouth, tongue, lips), which carries phonemes and emotional resonance.

Speech signal: $s[n] = e[n] * h[n]$ (convolution in time domain).

Taking the magnitude spectrum converts convolution into multiplication: $|S(\omega)| = |E(\omega)| \cdot |H(\omega)|$.

Taking the logarithm converts multiplication into addition:

$$\log|S(\omega)| = \log|E(\omega)| + \log|H(\omega)|$$

Applying the **Discrete Cosine Transform (DCT)** to the log Mel-energies separates low-frequency variations ($\log|H(\omega)|$, vocal tract shape) from high-frequency pitch ripple ($\log|E(\omega)|$):

$$\text{MFCC}[m] = \sum_{k=1}^{M} \log(Y[k]) \cdot \cos\left[m\left(k - \frac{1}{2}\right)\frac{\pi}{M}\right]$$

Taking the **first 20 to 40 cepstral coefficients** captures the vocal tract shape while discarding noise.

6. Dynamic Audio Features: Delta MFCCs (ΔMFCC)

Standard MFCCs capture static spectral shape per frame. To capture dynamic velocity (rate of pitch/tone change over time), we compute the first-derivative Delta coefficients:

$$d_t = \frac{\sum_{n=1}^{N} n(c_{t+n} - c_{t-n})}{2\sum_{n=1}^{N} n^2}$$

where c_t is the static MFCC coefficient at time frame t.

STAGE 2: Dataset Architecture & Preprocessing

1. Dataset Structural Breakdown

Your pipeline will combine two benchmark datasets:

1. **RAVDESS:** 1,440 files spoken by 24 professional actors (12 male, 12 female) across 8 emotions.

- Filename structure: `03-01-05-01-01-01-02.wav`
- 3rd digit represents Emotion ID (`01` = Neutral, `02` = Calm, `03` = Happy, `04` = Sad, `05` = Angry, `06` = Fearful, `07` = Disgust, `08` = Surprise).

2. **TESS:** 2,800 files spoken by 2 actresses across 7 emotions (missing *Calm*).

- Filename structure: `OAF_back_angry.wav` or `YAF_disgust.wav`

2. Audio Pre-processing Pipeline

Before feeding audio to feature extractors, apply signal conditioning:

- **Pre-Emphasis Filter:** Speech naturally attenuates at high frequencies. A high-pass FIR filter boosts high-frequency components:

$$y[n] = x[n] - \alpha \cdot x[n-1] \quad (\text{where } \alpha \approx 0.97)$$

```
import numpy as np
import librosa

def apply_pre_emphasis(signal, alpha=0.97):
    return np.append(signal[0], signal[1:] - alpha * signal[:-1])
```

- **Fixed Matrix Padding / Truncation:** Deep neural networks require uniform tensor dimensions. Audio files vary in duration (2–4 seconds).
- Set a fixed frame length $T_{\max} = 174$ frames (approx. 3 seconds of audio at $f_s = 16\ \text{kHz}$, frame length 512, hop length 160).
- If feature array frames $< T_{\max}$, zero-pad right.
- If feature array frames $> T_{\max}$, slice frames to $T_{\max}$.

STAGE 3: Deep Learning & Neural Network Operations

1. Convolutional Neural Networks (CNNs)

Unlike standard fully connected networks, CNNs use sliding weight matrices (kernels/filters) to extract localized spatial patterns invariant to position.

- **Convolution Operation:** For a 2D input image/spectrogram I and kernel K:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

- In **1D-CNNs**, the kernel slides along a 1D vector timeline.
- In **2D-CNNs**, the kernel slides simultaneously along time and frequency axes.
- **Activation Function (ReLU):** Introduces non-linearity so the network can learn complex emotional boundaries:

$$\text{ReLU}(z) = \max(0, z)$$

- **Max Pooling:** Reduces tensor dimensions while keeping dominant feature activations. A 2×2 Max Pool selects the maximum value in every non-overlapping 2×2 window.
- **Batch Normalization:** Normalizes internal activations per batch to have a mean of 0 and variance of 1. Accelerates training speed and prevents exploding/vanishing gradients.
- **Dropout Regularization:** During training, randomly deactivates a fraction p (e.g., 10%–30%) of neurons per forward pass. This prevents co-adaptation and overfitting.
- **Dense Layer + Softmax Output:** Maps learned feature representations down to 8 outputs corresponding to emotion probabilities:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{8} e^{z_j}}$$

STAGE 4: Project Implementation & Code

Step 1: Automated Data Loader & Feature Pipeline

This complete Python script scans directory paths, parses emotions, applies pre-emphasis, extracts features, and handles dynamic padding.

```
import os
import glob
import numpy as np
import librosa
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical

# Label Mapping for 8 Emotions
EMOTION_MAP = {
    'neutral': 0, 'calm': 1, 'happy': 2, 'sad': 3,
    'angry': 4, 'fearful': 5, 'disgust': 6, 'surprise': 7
}

def parse_ravdess_emotion(filename):
    parts = os.path.basename(filename).split('-')
    if len(parts) >= 3:
        code = parts[2]
        mapping = {'01':'neutral', '02':'calm', '03':'happy', '04':'sad',
                   '05':'angry', '06':'fearful', '07':'disgust',
'08':'surprise'}
        return mapping.get(code, None)
    return None

def parse_tess_emotion(filename):
    fname = os.path.basename(filename).lower()
    for emo in EMOTION_MAP.keys():
        if emo in fname:
            return emo
    if 'ps' in fname: # TESS uses 'ps' for pleasant surprise
        return 'surprise'
    return None

def extract_features_from_file(file_path, mode='2d_mfcc', n_mfcc=40,
max_pad_len=174):
    # Load audio at 16kHz
    y, sr = librosa.load(file_path, sr=16000)

    # 1. Pre-emphasis filter
    y_emp = np.append(y[0], y[1:] - 0.97 * y[:-1])
```

```
    if mode == '1d_mfcc_paper':
        # Baseline paper approach: 40 MFCCs averaged over time frames ->
(40, 1) vector
        mfcc = librosa.feature.mfcc(y=y_emp, sr=sr, n_mfcc=n_mfcc,
n_fft=512, hop_length=160)
        feat = np.mean(mfcc, axis=1) # Shape: (40,)
        return feat

    elif mode == '2d_mfcc_upgrade':
        # Novel upgrade approach: Full 2D spectrogram matrix -> (40,
max_pad_len, 1)
        mfcc = librosa.feature.mfcc(y=y_emp, sr=sr, n_mfcc=n_mfcc,
n_fft=512, hop_length=160)

        # Zero padding or truncating along time axis (axis 1)
        if mfcc.shape[1] < max_pad_len:
            pad_width = max_pad_len - mfcc.shape[1]
            mfcc = np.pad(mfcc, pad_width=((0,0), (0, pad_width)),
mode='constant')
        else:
            mfcc = mfcc[:, :max_pad_len]

        return mfcc[:, :, np.newaxis] # Expand dims for 2D-CNN: (40, 174,
1)

def load_dataset(dataset_dir, mode='2d_mfcc_upgrade'):
    X, y = [], []
    file_paths = glob.glob(os.path.join(dataset_dir, '**', '*.wav'),
recursive=True)

    for path in file_paths:
        emotion = parse_ravdess_emotion(path) or parse_tess_emotion(path)
        if emotion is None or emotion not in EMOTION_MAP:
            continue

        features = extract_features_from_file(path, mode=mode)
        X.append(features)
        y.append(EMOTION_MAP[emotion])

    X = np.array(X)
    y = to_categorical(np.array(y), num_classes=8)
    return X, y
```

Step 2: Baseline Model (1D-CNN) vs. Upgrade Model (2D-CNN)

Architecture 1: Paper Baseline (1D-CNN)

Recreates the exact multi-stage 1D convolution pipeline from Figure 8 in the paper:

```
import tensorflow as tf
from tensorflow.keras import layers, models

def build_paper_1d_cnn(input_shape=(40, 1), num_classes=8):
    model = models.Sequential([
        # Stage 1: Conv1D (64 filters) + MaxPool
        layers.Conv1D(64, kernel_size=5, activation='relu', padding='same',
input_shape=input_shape),
        layers.Dropout(0.10),
        layers.MaxPooling1D(pool_size=4),

        # Stage 2: Conv1D (128 filters)
        layers.Conv1D(128, kernel_size=5, activation='relu',
padding='same'),
        layers.Dropout(0.10),
        layers.MaxPooling1D(pool_size=4),

        # Stage 3: Conv1D (256 filters)
        layers.Conv1D(256, kernel_size=3, activation='relu',
padding='same'),
        layers.Dropout(0.10),

        # Dense Output
        layers.Flatten(),
        layers.Dense(512, activation='relu'),
        layers.Dropout(0.20),
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
    return model
```

Architecture 2: Novel Upgrade (2D-CNN for Spectrogram Matrices)

Preserves time-frequency patterns to achieve higher classification robustness:

```
def build_upgrade_2d_cnn(input_shape=(40, 174, 1), num_classes=8):
    model = models.Sequential([
        # Block 1: Feature Extraction
        layers.Conv2D(32, (3, 3), padding='same', input_shape=input_shape),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Dropout(0.25),

        # Block 2: Deep Pattern Recognition
        layers.Conv2D(64, (3, 3), padding='same'),
        layers.BatchNormalization(),
```

```
        layers.Activation('relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Dropout(0.25),

        # Block 3: Context Aggregation
        layers.Conv2D(128, (3, 3), padding='same'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Dropout(0.30),

        # Flatten & Dense Classification
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.40),
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
    return model
```

Step 3: Model Execution & Performance Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

# 1. Load Data
X, y = load_dataset('/path/to/ravdess_and_tess', mode='2d_mfcc_upgrade')

# 2. Split Data (67% Train, 33% Test matching paper methodology)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
random_state=42, stratify=np.argmax(y, axis=1))

# 3. Instantiate & Train Model
model = build_upgrade_2d_cnn(input_shape=X_train.shape[1:])
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=100,
    batch_size=32
)

# 4. Generate Confusion Matrix & Validation Plots
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)
```

```
# Plot Confusion Matrix
cm = confusion_matrix(y_true_classes, y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=list(EMOTION_MAP.keys()),
            yticklabels=list(EMOTION_MAP.keys()))
plt.title('Confusion Matrix - 2D CNN Upgrade')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# Print Precision, Recall, F1-Score Report
print(classification_report(y_true_classes, y_pred_classes,
target_names=list(EMOTION_MAP.keys())))
```

Checklist for Paper & Project Deliverables

When submitting your final Signals & Systems project report, ensure you include:

1. **Theoretical Signal Processing Section:** Include equations for Hamming windowing, Discrete Fourier Transform (FFT), Mel Scale conversion, and Discrete Cosine Transform (MFCC).
2. **Comparative Experimental Table:** Compare performance across configurations:

- **Paper Baseline (1D MFCC Vector + 1D-CNN):** 95.24% train/93% test.
- **Paper Multi-feature (MFCC + Delta + Chroma + Mel-Spec):** 87.58% train/82% test (note performance drop due to feature redundancy).
- **Your Novel Upgrade (Full 2D MFCC Matrix + 2D-CNN):** Measure accuracy gains achieved by preserving time-frequency dynamics.

3. **Confusion Matrix Analysis:** Discuss class-wise performance (e.g., explaining why *Calm* and *Sad* show higher confusion due to similar pitch and low energy levels).